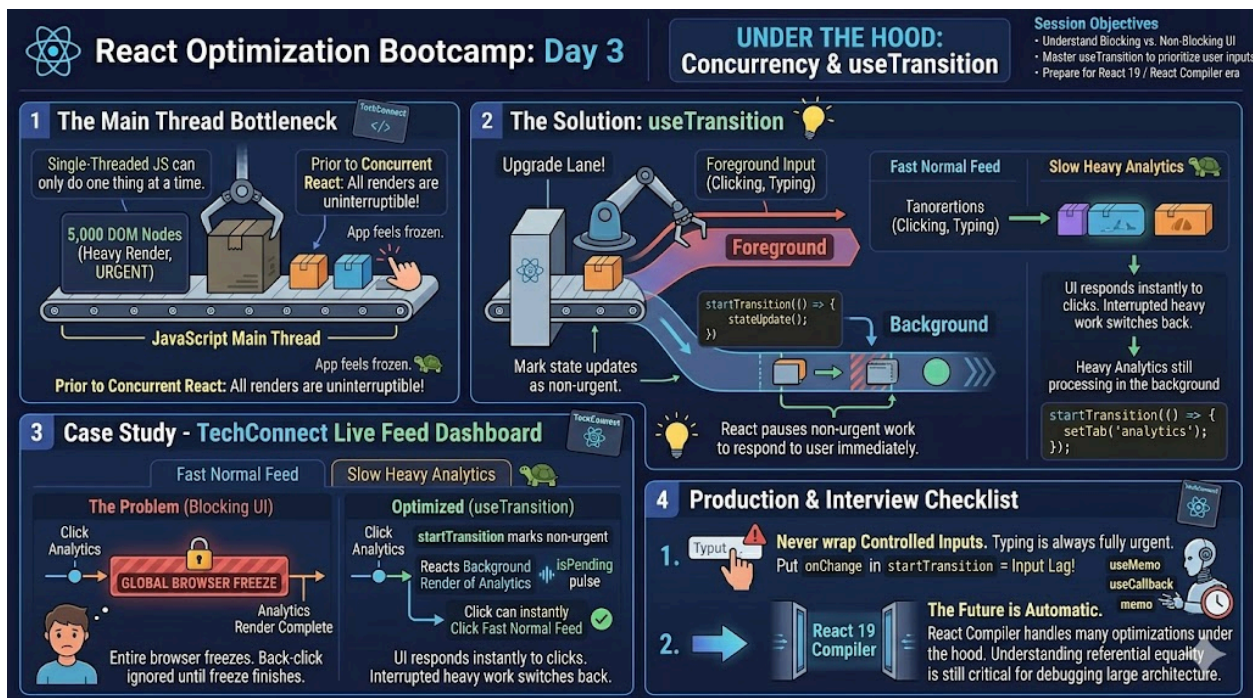


Day 3: Concurrent React

(`useTransition`)

🎯 Objectives

- Understand the concept of "Blocking" vs. "Non-Blocking" UIs.
- Master `useTransition` to prioritize user inputs (like clicking and typing) over heavy data rendering.
- Prepare for the React 19 / React Compiler era.



🧠 The Main Thread Bottleneck

JavaScript is single-threaded. This means the browser can only do one thing at a time. If React is busy rendering 5,000 DOM nodes for a massive chart, the browser cannot listen to user clicks, scrolling, or typing. The app feels "frozen."

Prior to Concurrent React, all renders were "urgent." Once React started rendering, it could not be interrupted.



useTransition

`useTransition` allows you to mark specific state updates as **non-urgent**.

It tells React: *"You can start working on this heavy render in the background, but if the user clicks something else or starts typing, pause the heavy work and respond to the user immediately."*

TechConnect Live Feed

Today, we tackled the Live Feed dashboard.

The Problem: We have a "Normal Feed" tab and a heavily intensive "Analytics" tab. When a user clicked "Analytics", the entire browser froze for several seconds while it processed. If they tried to quickly click back to "Normal Feed", nothing happened until the freeze finished.

The Optimized Solution

We wrapped the tab switching logic in `startTransition`. Now, the UI instantly responds to clicks, and we can show a smooth loading indicator without freezing the main thread.

JavaScript

```
import React, { useState, useTransition, memo } from 'react';

export default function LiveFeedDay3() {
  const [tab, setTab] = useState('feed');
  const [isPending, startTransition] = useTransition();

  // OPTIMIZATION: startTransition
  // This marks the state update as non-urgent. The UI remains fully responsive.
  // If the user clicks "Heavy Analytics" and immediately clicks "Normal Feed",
  // React interrupts the heavy render and instantly switches back!
  const selectTab = (nextTab) => {
    startTransition(() => {
      setTab(nextTab);
    });
  };

  return (
    <div className="p-8 bg-white rounded-2xl shadow-sm border border-slate-200">
      <h2 className="text-2xl font-bold text-slate-900 mb-6">Live Feed (Day 3)</h2>

      <div className="flex space-x-2 mb-6">
        <TabButton isActive={tab === 'feed'} onClick={() => selectTab('feed')}>
          Normal Feed
        </TabButton>
      </div>
    </div>
  );
}
```

```

      </TabButton>
      <TabButton isActive={tab === 'analytics'} onClick={() => selectTab('analytics')}>
        Heavy Analytics
      </TabButton>
    </div>

    <div className="min-h-[400px] border-t border-slate-100 pt-6">
      /* We can use 'isPending' to show a subtle indicator without unmounting the old UI
    */}
      {isPending && (
        <p className="text-indigo-600 font-bold mb-4 animate-pulse">
          Processing Analytics in background...
        </p>
      )}
      {tab === 'feed' && <NormalFeed />}
      {tab === 'analytics' && <SlowAnalytics />}
    </div>
  </div>
);
}

const TabButton = ({ children, isActive, onClick }) => (
  <button
    onClick={onClick}
    className={`px-4 py-2 rounded-lg font-medium transition-colors ${
      isActive ? 'bg-indigo-600 text-white shadow-md' : 'bg-slate-100 text-slate-600
hover:bg-slate-200'
    }`}
  >
    {children}
  </button>
);

const NormalFeed = () => (
  <div className="p-4 bg-slate-50 rounded-lg text-slate-600">
    This is a fast, normal component. Clicking back here is instant.
  </div>
);

// INTENTIONAL BOTTLENECK: Simulates a massive data grid or complex chart
const SlowAnalytics = memo(() => {
  console.log("Rendering Extremely Slow Component... 🐢");

  const items = [];
  for (let i = 0; i < 500; i++) {
    // Artificial heavy computation blocking the thread
    let startTime = performance.now();
    while (performance.now() - startTime < 2) {}
    items.push(<div key={i} className="h-2 w-full bg-slate-200 mb-1 rounded"></div>);
  }
}

```

```

return (
  <div>
    <h3 className="text-lg font-bold text-rose-600 mb-4 flex items-center">
      <span className="w-2 h-2 rounded-full bg-rose-600 animate-ping mr-2"></span>
      Heavy Analytics Loaded
    </h3>
    <div className="grid grid-cols-5 gap-2">
      {items}
    </div>
  </div>
);
});

```



Production & Interview Checklist

- **Never wrap Inputs in Transitions:** Do not put `onChange` handlers for controlled text inputs inside `startTransition`. Typing should always be fully synchronous and urgent.
- **The Future is Automatic:** As we look toward React 19 and the React Compiler, many of the manual memoization hooks (`useMemo`, `useCallback`, `React.memo`) will be handled automatically under the hood. However, understanding the underlying rules of referential equality and thread blocking remains critical for debugging large-scale architecture.